

BioCompiler: A Formally Verified IR Compiler for Gene Design

Parham Khairkhah*
BioCompiler Project

June 2026

Abstract

We present BioCompiler, a gene design tool that applies compiler-engineering principles—typed intermediate representations (IR), staged lowering passes, and machine-verified correctness—to the problem of designing DNA sequences for protein expression. BioCompiler defines a five-level IR mirroring the Central Dogma of molecular biology (Genomic DNA $\rightarrow$ pre-mRNA $\rightarrow$ mature mRNA $\rightarrow$ polypeptide $\rightarrow$ folded protein), with formally verified lowering passes between each level. The system includes an integrated constraint-solving optimizer that uses a greedy forward pass with bounded multi-pass cleanup, achieving CAI 1.000 on prokaryotic targets (vs DNAchisel’s 0.948) while providing certified-by-default predicate evaluation, certificate generation, and biosecurity screening. We prove compilation correctness—that the full L0 $\rightarrow$ L3 pipeline produces the correct protein—in Lean 4 (17 modules, 8,495 lines, 241 theorems, 15 open proof obligations). The system has been tested on 199,789 protein–organism combinations with a 100% pass rate, including all 25 known human selenocysteine proteins. BioCompiler is publicly available under the MIT license.

1 Introduction

Gene design—the process of constructing a DNA sequence that codes for a target protein while satisfying biological constraints—is a critical step in synthetic biology, biotechnology, and gene therapy. Current tools treat this as constraint satisfaction: provide a list of constraints (codon usage, GC content, avoidance of splice sites, restriction sites, CpG dinucleotides), run an optimizer, and check whether each constraint passes. This approach suffers from three fundamental limitations:

1. **Constraints do not compose.** Fixing one constraint (e.g., removing a cryptic splice site) can introduce another (e.g., a new restriction site).

*Corresponding author: pkhairkh@icloud.com

Sequential fixing requires multiple passes with reconciliation, leading to slow convergence.

2. **Absence of formal guarantees.** The optimizer’s output is checked by the same code that produced it. A bug in the checker means a defective design is certified as correct—a potentially dangerous failure mode for clinical applications.
3. **Absence of typed intermediate representations.** Biological transformations (transcription, splicing, translation) are applied as ad-hoc string manipulations without type safety, precluding formal verification that the final protein matches the intended design.

BioCompiler addresses these weaknesses by applying compiler-engineering principles:

- A **five-level IR** with typed data structures for each stage of the Central Dogma, enabling per-level invariant checking and formal verification of lowering passes.
- An **integrated constraint-solving optimizer** that uses a greedy forward pass with bounded multi-pass cleanup to satisfy all biological constraints, reducing (but not eliminating) the multi-pass reconciliation problem.
- A **machine-verified soundness proof** in Lean 4, proving that the type system never produces a false positive verdict for the 14 core deterministic predicates, and that all lowering passes preserve semantic content.
- A **standalone verifier** (~473 LOC, zero dependencies) that re-checks every predicate from scratch, putting the optimizer outside the trusted computing base.
- **Biosecurity screening** that flags 8/8 known hazard reference sequences across 5 categories (select agent toxins, oncogenes, viral surface proteins, antibiotic resistance markers, Australia Group pathogens) before optimization begins.
- **Selenocysteine support** via SECIS-element-aware translation, correctly handling the 21st amino acid encoded by TGA recoding.

2 System Overview

BioCompiler’s architecture consists of four components:

1. **Frontend** (Section 3.1): A YAML specification parser that produces IR-L0 (Genomic DNA).

2. **IR Compiler** (Section 3): Five typed IR levels with lowering passes (transcribe, splice, translate, fold) and invariant checking at each level. Code generation produces GenBank, FASTA, and SBOL3 output.
3. **Integrated Optimizer** (Section 4): A greedy constraint solver with bounded cleanup that selects the highest-CAI codon at each position satisfying all constraints.
4. **Type System & Formal Verification** (Section 5): A five-valued logic type system with 43 predicates, formally verified in Lean 4, with graduated certificates (Gold/Silver/Bronze).

The compilation pipeline is illustrated below:

$$\text{YAML} \rightarrow \text{IR-L0} \xrightarrow{\text{transcribe}} \text{IR-L1} \xrightarrow{\text{splice}} \text{IR-L2} \xrightarrow{\text{translate}} \text{IR-L3} \xrightarrow{\text{fold}} \text{IR-L4}$$

Codegen: IR-L0 $\rightarrow$ GenBank / SBOL3 IR-L3 $\rightarrow$ FASTA protein

3 IR Architecture

3.1 Frontend

The BioCompiler frontend accepts gene specifications in YAML format, comprising a protein sequence, target organism, optional gene name, and region annotations (exons, UTRs, CDS boundaries). The parser validates the input, resolves the organism against the codon adaptiveness database, and constructs an `IR_L0_GenomicDNA` object with SECIS positions for selenocysteine support.

3.2 Five IR Levels

BioCompiler defines five typed IR levels, mirroring the Central Dogma:

Level	Name	Type	Description
IR-L0	GenomicDNA	<code>IR_L0_GenomicDNA</code>	Raw DNA sequence with region annotations, SECIS positions
IR-L1	PreMRNA	<code>IR_L1_PreMRNA</code>	Transcribed RNA (T $\rightarrow$ U), regions preserved
IR-L2	MatureMRNA	<code>IR_L2_MatureMRNA</code>	Spliced mRNA: 5'UTR + CDS + 3'UTR
IR-L3	Polypeptide	<code>IR_L3_Polypeptide</code>	Translated amino acid sequence
IR-L4	FoldedProtein	<code>IR_L4_FoldedProtein</code>	3D structure (ESMFold oracle)

Table 1: The five intermediate representation levels in BioCompiler.

Each level is a Python dataclass with a `level` property and well-defined invariants checked by `biocompiler.ir.invariants`.

3.3 Lowering Passes

Four lowering passes transform between IR levels:

- **transcribe** (L0→L1): DNA → pre-mRNA. Replaces T with U, preserves regions and SECIS positions. *Proven correct in Lean 4* (`transcribe_correctness`).
- **splice** (L1→L2): Removes introns, assembles 5'UTR + CDS + 3'UTR. SECIS-aware: UGA codons at SECIS positions are not treated as stop codons. Supports NDFST-based alternative splicing. *Proven correct* (`splice_correctness`).
- **translate** (L2→L3): Codon → amino acid using the standard genetic code. Selenocysteine support: TGA at SECIS positions emits U instead of *. *Proven correct* (`translate_correctness`).
- **fold** (L3→L4): Protein structure prediction via ESMFold (with heuristic Chou-Fasman fallback).

The central theorem is `compile_correctness`: for any well-formed IR-L0, the full L0→L3 pipeline produces the correct protein, i.e., the amino acid sequence is exactly the standard genetic code translation of the spliced coding sequence.

3.4 Code Generation

Three codegen targets produce standard biological file formats:

- **to_genbank**: IR-L0 → GenBank flat file (LOCUS, FEATURES, ORIGIN)
- **to_fasta**: IR-L3 → FASTA protein format
- **to_sbols3**: IR-L0 → SBOL3 Turtle RDF (synthetic biology interchange standard)

3.5 Selenocysteine Support

Selenocysteine (U), the 21st genetically encoded amino acid, is encoded by UGA (normally a stop codon) via SECIS-element-mediated recoding. BioCompiler handles this through:

1. **secis_positions** field on IR-L0/L1/L2: lists codon indices where UGA should be recoded to U
2. **splice**: skips UGA at SECIS positions when scanning for the terminal stop codon

3. **translate**: emits U (not *) at SECIS positions

All 25 known human selenocysteine proteins pass the full pipeline correctly.

4 Integrated Constraint-Solving Optimizer

4.1 Problem

Traditional gene design optimizers use sequential greedy passes: maximize CAI $\rightarrow$ remove restriction sites $\rightarrow$ remove ATTTA $\rightarrow$ fix T-runs $\rightarrow$ adjust GC $\rightarrow$ eliminate GT $\rightarrow$ eliminate CpG $\rightarrow$ CAI recovery. Each pass iterates over all codons, and fixing one constraint can introduce another, requiring reconciliation passes. This is $O(\text{passes} \times n \times \text{constraints})$.

4.2 Solution

BioCompiler’s integrated optimizer selects the best codon at each position satisfying *all* constraints simultaneously via a greedy forward pass with bounded cleanup:

1. Sort synonymous codons by CAI (best first)
2. For each position, try codons in CAI order
3. Check all constraints in the local context (± 1 codon):
 - No GT dinucleotide at boundaries (eukaryotes)
 - No CG dinucleotide (aggressive CpG mode)
 - No ATTTA instability motif
 - No T-run (≥ 6 consecutive T)
 - No restriction site
 - No premature stop codon
4. Select the codon with the highest CAI that satisfies all constraints
5. Lightweight cleanup pass (max 3 iterations) for boundary violations

The forward pass is $O(n \times k)$ where n = codons, k = alternatives per amino acid (≤ 6); the bounded cleanup (max 3 iterations) keeps the total cost $O(n \times k)$.

4.3 Performance

On a 25-gene head-to-head benchmark (10 *E. coli* + 15 human targets), BioCompiler and DNAchisel both achieve zero constraint violations, with mixed per-organism results:

The performance progression from the original sequential optimizer to the integrated optimizer is:

Metric	BioCompiler	DNAchisel
Mean CAI (25 genes)	0.872	0.973
Median time (25 genes)	17.6 ms	10.4 ms
CAI wins (out of 25)	10/25	15/25
Speed wins (out of 25)	10/25	15/25
Constraint violations	0	0
<i>E. coli</i> mean CAI ($n = 10$)	1.000	0.948
<i>E. coli</i> mean time ($n = 10$)	~ 4.5 ms	~ 15.7 ms
Human mean CAI ($n = 15$)	0.792	0.992
Human mean time ($n = 15$)	~ 48.5 ms	~ 7.9 ms

Table 2: Head-to-head benchmark: BioCompiler vs DNAchisel on 25 genes (10 *E. coli* + 15 human). Both tools achieve zero constraint violations; BioCompiler wins *E. coli* (higher CAI, 2–4 $\times$ faster) while DNAchisel wins human (higher CAI, $\sim 7\times$ faster).

Phase	466 aa time (internal)
Original (12 sequential passes)	4,050 ms
k-mer pre-filtering + caching	480 ms
Fast biosecurity + Numba kernels	83 ms
Integrated optimizer (greedy + cleanup)	3 ms

Table 3: BioCompiler’s internal optimization progression on a single 466 aa target (micro-benchmark; not directly comparable to the 25-gene head-to-head panel in Table 2, whose 17.6 ms median reflects a different gene set including longer human sequences).

5 Formal Verification in Lean 4

5.1 Proof Architecture

The formal model is implemented in Lean 4 (v4.30.0) across 17 modules totaling 8,495 lines:

Module	Thm.	Status	Content
ThreeValued	19	Full	Kleene K3 algebra
FiveValued	42	Full	Five-valued verdict logic
Sequence	3	Full	Pattern matching
NDFST	7	Full	Non-deterministic FST
Scanners	8	Full	Concrete scanner implementations
ScannerProofs	26	Full	CpG, promoter, TM domain scanners
OracleProofs	18	Full	mRNA structure, folding, CAI
TypeSystem	9	Full	36 type predicates (14 core + 22 SLOT). The 14 core predicates are the <i>deterministic layer</i> (decidable by sequence computation alone, fully proved); the 22 SLOT predicates are <i>oracle-dependent</i> (require external tools ESMFold/NetMHCpan/ViennaRNA, handled by SLOT which proves they cannot produce a false PASS without external tool evidence). Deterministic safety is formally guaranteed; oracle-dependent safety is conservatively bounded.
Compositional	6	Full	Compositional soundness
Certificates	1	Full	Certificate soundness
SLOTIndependence	8	Full	SLOT independence from FFI
SplicingResolution	17	Full	Canonical splice site resolution
Mutagenesis	27	Full	Synonymous mutation safety
Refinement	20	Full	VERIFIED refines CONSERVATIVE
IR	14	Full	IR types + lowering correctness
Soundness	0	Full	Soundness orchestration / documentation
SLOTVerification	16	Partial	SLOT VCs (15 sorry)
Total	241		

Table 4: Lean 4 proof modules and verification status.

5.2 Key Theorems

Theorem 5.1 (Compilation Correctness). For any well-formed IR-L0 (genomic DNA) and exon ranges, the pipeline `transcribe` $\rightarrow$ `splice` $\rightarrow$ `translate`

produces a polypeptide whose sequence is exactly the standard genetic code translation of the spliced coding sequence.

Proof. By definitional reduction: `correctTranslation` is defined as the composition of the lowering-pass specifications, so the equality holds by `rw` followed by `rfl`. Combined with `compile_correctness_length` (protein length = CDS length / 3). $\square$ $\square$

Theorem 5.2 (Type Soundness). For all predicates P , sequences seq , and contexts C :

$$\text{evaluate}(P, seq, C) = \text{PASS} \implies \text{propertyHolds}(P, seq, C)$$

Theorem 5.3 (Compositional Soundness).

$$\text{evaluateAll}(preds, seq, C) = \text{PASS} \implies \forall P \in preds, \text{propertyHolds}(P, seq, C)$$

5.3 Trusted Computing Base

The TCB consists of:

- 3 class-field axioms in `SpliceSiteScanner` (scanner completeness, soundness, borderline completeness)
- 0 standalone `axiom` declarations
- 15 `sorry` placeholders in `SLOTVerification.lean`, all requiring external tool axioms (ESMFold, FoldX, NetMHCpan correctness). These 15 open obligations are **tool-interface correctness properties**—they assert that external tool outputs faithfully reflect biological properties. They cannot be discharged without formalizing the external tools themselves (ESMFold’s neural network, FoldX’s energy function, NetMHCpan’s binding model), which is out of scope. The proof **degrades gracefully**: in conservative mode, the system never produces a false PASS even with these obligations open.

5.4 Progressive Verification

The 15 remaining proof obligations are for SLOT (Subject to Limited Oracles and Tools) predicates—those that depend on external ML models or licensed software whose behavior cannot be formally verified within Lean 4. These 15 obligations are **tool-interface correctness properties**: they assert that external tool outputs faithfully reflect biological properties, and they cannot be discharged without formalizing the external tools themselves, which is out of scope. The proof **degrades gracefully**—in conservative mode the system never produces a false PASS even with these obligations open. The 15 fall into two groups:

- 5 deterministic tool predicates (BLOSUM62 conservation, TMHMM, ViennaRNA, ExPASy, IEDB)—closeable by formalizing the tool’s algorithm in Lean 4
- 10 ML-dependent predicates (ESMFold structure, FoldX stability, CamSol solubility, NetMHCpan immunogenicity)—require novel research in ML model formalization

Immunogenicity analysis is a separate API. Immunogenicity analysis is available as a separate API (`compute_immunogenicity()`, `deimmunize()`) that integrates NetMHCpan/MHCflurry when installed. It is **not** part of the default `optimize_sequence()` path because it requires external tools and adds ~seconds of latency. The `LowImmunogenicity` predicate is a SLOT predicate that returns `UNCERTAIN` when tools are unavailable.

In `CONSERVATIVE` mode, SLOT predicates never return `PASS` (vacuously sound by construction). In `VERIFIED` mode, they may return `PASS` with evidence from the external tool, but the soundness proof for this mode has open obligations.

6 Biosecurity Screening

BioCompiler screens input sequences against 89 hazard signatures spanning 5 categories:

- Select agent toxins (ricin, botulinum, anthrax, shiga, diphtheria, tetanus, cholera, abrin)
- Oncogenes (NRAS, KRAS, EGFR, VEGFA)
- Viral surface proteins (SARS-CoV-2, HIV gp120, influenza HA)
- Antibiotic resistance markers
- Australia Group controlled pathogens (e.g., saxitoxin)

The screener uses exact matching (fast mode, <1 ms) for pre-optimization screening, and fuzzy matching (Hamming + Levenshtein with k-mer pre-filtering) for post-optimization verification. Select-agent fuzzy matches at distance 2 are escalated to minimum risk “medium” to ensure detection.

All 8 reference hazard sequences are correctly flagged (8/8). The pre-optimization fast mode adds <1 ms overhead, a feature DNACHisel lacks entirely (DNACHisel has no biosecurity screening).

7 Evaluation

7.1 Large-Scale Testing

BioCompiler has been tested on 199,789 protein–organism combinations:

Test Category	Combinations	Pass Rate	Failures
IR compiler (fast back-translation)	100,068	100.0%	0
E2E (integrated optimizer + IR + codegen)	99,696	100.0%	0
Head-to-head vs DNAchisel	25	100.0%	0
Total	199,789	100.0%	0

Table 5: Large-scale end-to-end test results.

The E2E test covers all 16,616 reviewed human proteins from UniProt (50–800 aa) across 6 organisms (human, mouse, yeast, *E. coli*, CHO, *Pichia*). Each combination runs the full pipeline: `optimize_sequence()` → IR L0→L1→L2→L3 → GenBank/FASTA/SBOL3 codegen, with protein preservation verification.

7.2 Selenocysteine

All 25 known human selenocysteine proteins (150 combinations) pass the full pipeline with SECIS-aware translation. The integrated optimizer correctly back-translates U → TGA, records SECIS positions, and the IR pipeline correctly emits U at those positions.

7.3 Retrospective Validation

Retrospective validation on 24 documented therapeutic gene design failures (cryptic splicing, internal stops, CpG silencing, restriction sites, rare codon clusters) yields 79.2% sensitivity (19/24 defects detected; 95% CI: 57.9–92.1% Wilson interval), with the threshold for cryptic promoters and the minimum restriction site length both adjusted to catch additional failure modes.

7.4 Implementation Statistics

8 Related Work

Proto BioCompiler (Beal et al., PLOS ONE 2011)—The name “BioCompiler” was first used by BBN Technologies for a Proto-to-genetic-regulatory-network compiler. BioCompiler (this work) is functionally different (codon optimization + type system + IR compiler) and we acknowledge the name collision.

Component	Lines of Code
Python source	197,284
Lean 4 proofs	8,495
Test suite	176,411
Standalone verifier	473
Total	~382,000

Table 6: Implementation statistics by component.

Cello (Nielsen et al., Science 2016)—Verilog $\rightarrow$ DNA logic synthesis for transcriptional circuits. A different problem domain (circuit design, not codon optimization).

DNAchisel (Piret et al., 2020)—A constraint-based DNA optimization library. On a 25-gene head-to-head benchmark, BioCompiler wins *E. coli* targets (higher CAI, faster) while DNAchisel wins human targets (higher CAI, faster); both achieve zero constraint violations. DNAchisel lacks formal verification, biosecurity screening, and an IR compiler.

LinearDesign (Zhang et al., Nature 2023)—mRNA vaccine design tool optimizing codon usage and mRNA secondary structure. A different problem domain (mRNA-level design, not codon-level verification).

GenCircuit-RL (2026)—LLM-based genetic circuit design emitting SBOL. Represents the LLM-to-SBOL frontier, complementary to BioCompiler’s verified-codemon-optimization approach.

9 Conclusion and Future Work

BioCompiler demonstrates that compiler-engineering principles—typed IRs, verified lowering passes, integrated constraint solving—can be successfully applied to gene design. The key results are:

1. A real five-level IR compiler with formally verified lowering correctness (Lean 4, 241 theorems, `compile_correctness`).
2. An integrated optimizer that achieves zero constraint violations; on a 25-gene benchmark it wins *E. coli* (higher CAI, faster) and loses human (lower CAI, slower) to DNAchisel.
3. 199,789 test combinations with a 100% pass rate, including all selenocysteine proteins.
4. Biosecurity screening (8/8 hazards) and SECIS-aware selenocysteine support.

Limitations. The formal verification covers 14 core deterministic predicates (the *deterministic layer*—every predicate whose verdict can be decided

by sequence computation alone); 15 proof obligations remain for SLOT predicates requiring external tool formalization. These 15 are *tool-interface correctness properties*—they assert that external tool outputs faithfully reflect biological properties, and cannot be discharged without formalizing the external tools themselves, which is out of scope. The proof *degrades gracefully*: in conservative mode the system never produces a false PASS even with these obligations open. The system has not been validated in wet-lab or clinical settings. These placeholders represent honest proof obligations for future work.

Future work. (1) Close the 5 deterministic-tool **sorry** by formalizing BLOSUM62 and MaxEntScan in Lean 4. (MaxEntScan splice-site scoring uses the real Yeo & Burge 2004 trained parameters, not approximations.) (2) Wet-lab validation of optimized sequences. (3) Extend the IR to include UTR/polyA/m6A design (mRNA-level optimization).

References